

MODULE IV: UI/UX DESIGN AND PROTOTYPING FOR LOW-CODE / NO-CODE APPLICATIONS

1. Introduction to Figma for LCNC Applications

- Figma is a cloud based UI design and prototyping tool that runs in the browser (and as a desktop app) and stores all design files online.
- Teams can design screens, create prototypes, and collaborate in real time on the same file, which makes it ideal for low code / no code (LCNC) application projects where business users, designers, and developers work together.

For LCNC applications, Figma is typically used to:

- Design screen layouts and flows before building in tools like Bubble, Adalo, or other LCNC platforms.
- Define design systems (colors, typography, components) that can be consistently implemented using LCNC visual components.
- Prototype and test user flows early, so that the LCNC build focuses on validated designs and reduces rework.

A typical LCNC workflow with Figma:

1. Gather requirements and user stories.
2. Create low-fidelity wireframes in Figma (basic structure and flow).
3. Refine into high-fidelity UI with colors, typography, and components.
4. Add interactions and user flows as Figma prototypes.

2. UI vs UX: Distinction

UI (User Interface) and UX (User Experience) are related but distinct concepts in application design.

- UI is about the **visual** and interactive elements the user sees and touches: colors, typography, buttons, forms, icons, spacing, and layout.
- UX focuses on the overall **experience**: how easy it is to accomplish tasks, how intuitive the flow is, how users feel while using the product, and whether their goals are met efficiently.

In LCNC projects, both UI and UX are critical because:

- Poor UI can make even a well-structured app look unprofessional and confusing.
- Poor UX leads to user drop-off, increased support costs, and business failure, even if the UI looks visually appealing.

3. Design Thinking Basics

Design thinking is a human-centered problem-solving approach that emphasizes understanding users, exploring many ideas, and iteratively testing solutions.

It is commonly described as a process with five phases:

1. Empathize:

- Understand users' needs, pain points, context, and behaviors through interviews, observation, and surveys.
- For LCNC apps, this may involve talking to domain experts (e.g., HR managers for an HR app) and end-users.

2. Define:

- Synthesize findings into clear problem statements (e.g., “Employees struggle to submit expense claims quickly on mobile”).
- Translate into user stories or requirements to drive designs.

3. Ideate:

- Generate many possible ideas and UI concepts without judging them initially.
- Use sketches and low-fidelity wireframes in Figma to explore layouts and flows.

4. Prototype:

- Build interactive prototypes in Figma that simulate the app's behaviour.
- Focus on core flows (login, main task flows) to validate concepts quickly before implementing in LCNC tools.

5. Test:

- Conduct usability tests with real users on Figma prototypes.
- Collect feedback, identify usability problems, and iterate on the design.

4. Accessibility Principles (WCAG Basics)

The Web Content Accessibility Guidelines (WCAG) provide a globally recognized standard for making digital content accessible to people with disabilities.

WCAG is organized under four core principles often remembered as POUR:

1. Perceivable:

- Information and UI components must be presentable to users in ways they can perceive.
- Examples: Providing text alternatives (alt text) for images, ensuring sufficient color contrast between text and background, providing captions for videos.

2. Operable:

- UI components and navigation must be operable through various input methods, including keyboard.
- Examples: Keyboard-accessible controls, focus indicators, avoiding content that flashes excessively, providing enough time for interactions.

3. Understandable:

- Information and the operation of the UI must be understandable.
- Examples: Clear labels and instructions, predictable navigation and behavior, error messages that explain what went wrong and how to fix it.

4. Robust:

- Content must be robust enough to be reliably interpreted by a wide variety of user agents, including assistive technologies.
- Examples: Using semantic HTML structures, proper form field associations, and avoiding invalid markup.

5. Cloud-Based Design and Real-Time Collaboration

Figma is a fully cloud-based tool, meaning:

- Files are stored on Figma's servers and accessed via the browser or app.
- Multiple team members can open the same file simultaneously, see each other's cursors, and edit together in real time.

Important collaboration features:

- Real-time co-editing: Designers, product managers, and developers can work in parallel on layout, content, and comments without file locking.
- Commenting and feedback: Stakeholders can leave comments directly on frames or components to specify changes or ask questions.
- Shareable links: Designers can share view or edit links with access control (e.g., link with “can view” or “can edit”).

For LCNC projects, cloud-based collaboration helps:

- Non-technical stakeholders visualize the app before any LCNC implementation.
- Distributed teams synchronize on the same source of truth (single design file).
- Rapid iteration cycles, because feedback loops are shorter and more visible.

6. Understanding the Figma Interface, Workspace, and Workflow

The Figma interface consists of several key areas:

- **Top bar:**
 - File name, sharing options, zoom controls, play (present) for prototypes, and device preview options.
- **Left panel:**
 - Layers panel: Shows frames, groups, and components in a hierarchical tree.
 - Assets panel: Shows reusable components and styles from local and team libraries.

- **Canvas:**

- Main workspace where frames, shapes, and screens are placed and edited.

- **Right panel:**

- Design tab: Properties like position, size, constraints, auto layout, fills, strokes, effects, and text properties.
- Prototype tab: Interaction triggers, actions, destinations, and animation settings.
- Inspect tab: Code and specs for developers (CSS, iOS, Android values).

7. Frames, Layouts, Grids, and Alignment

Frames

In Figma, frames are containers that hold other elements like shapes, text, and components.

They are used to represent:

- Individual screens (e.g., mobile app screen, web page).
- Layout sections (e.g., header, content area, sidebar).
- Components (e.g., button, card) when combined with component features.

Frames support:

- Constraints (defining how child elements respond to resizing).
- Auto layout (automatic arrangement of items).
- Background fills and fixed dimensions.

Layout Grids

Layout grids provide structure and alignment for responsive and consistent layouts.

Types include:

- Columns: Commonly used for multi-column responsive designs (e.g., 12-column web grid).
- Rows: Useful for vertical spacing and section grouping.
- Grid: Uniform square grid for icons or more detailed pixel alignment.

Using layout grids:

- Helps maintain consistent margins and spacing.
- Facilitates responsive design thinking for LCNC implementations.
- Makes handoff easier, as developers can map columns and spacing to LCNC layout controls.

Alignment and Distribution

Figma offers alignment tools in the top bar or right panel:

- Align left, center, right, top, middle, bottom.
- Distribute horizontally or vertically to equalize spacing.

Good alignment practices:

- Align labels and form elements for easy scanning.
- Use consistent margins between sections.
- Avoid “pixel-hunting” by using alignment tools instead of manual positioning.

8. Creating and Styling UI Components

Shapes and Basic Elements

Common shapes in Figma:

- Rectangle, line, ellipse, polygon, star, and others.
- These shapes can represent buttons, cards, images, icons, and backgrounds.

Properties you can set:

- Fill: Solid colors, gradients, or images.
- Stroke: Borders with thickness, color, and corner styles.
- Corner radius: Rounded corners, per-corner control.

Text and Typography

Text is central to usability and clarity. In Figma:

- Text properties include font family, size, weight, line height, letter spacing, alignment, and decoration.
- Designers often define text styles (e.g., Heading 1, Heading 2, Body, Caption) to ensure consistency.

Typography guidelines for LCNC app screens:

- Use a small set of type styles to maintain visual hierarchy.
- Ensure text is legible on small mobile screens (common body sizes around 14–16 px for mobile).
- Use contrast and size to differentiate headings from body text.

Colors

Color styles can be defined and reused across the file:

- Primary colors (e.g., brand color).
- Secondary and accent colors.
- Neutral palette (backgrounds, borders, text).

Accessibility considerations:

- Sufficient contrast between text and background (for normal text, WCAG recommends a minimum contrast ratio of 4.5:1 for Level AA; for large text, 3:1).
- Avoid relying solely on color to convey information; also use icons or labels.

9. Components, Variants, and Design Systems

Components

Components are reusable UI elements that maintain a master–instance relationship:

- Master (main component): The source definition that controls default appearance and structure.
- Instances: Copies placed on screens that inherit properties but can override specific details (e.g., text label).

Use components for:

- Buttons, input fields, cards, navigation bars, dialogs, and icons.
- Repeating patterns that appear in multiple screens.

Benefits:

- Consistency across the app.
- Faster updates: changing the master updates all instances.
- Clear mapping to LCNC platform components (e.g., “primary button” component to LCNC “button” widget).

Variants

Variants allow multiple related component states to be grouped as a single component set.

Examples:

- Button variants: primary, secondary, disabled, with icon, loading.
- Input field variants: default, focused, error, success.

Variant properties:

- Designers can define properties like “State”, “Type”, “Size” and switch between them in the properties panel.

Design Systems

A design system is a collection of reusable components, styles, and guidelines that define the visual and interaction language of a product.

In Figma, design systems typically include:

- Color styles and text styles.
- Component libraries (buttons, forms, navigation, cards).
- Grid and spacing guidelines.
- Documentation on usage.

For LCNC applications:

- The design system in Figma acts as a blueprint that LCNC builders follow when configuring their components and themes.
- It reduces inconsistency and makes scaling the application (e.g., adding more modules) more manageable.

10. Auto Layout for Responsive Design

Auto Layout in Figma automatically arranges elements inside a frame according to direction, spacing, padding, and alignment settings.

When content changes, the layout updates automatically without manual adjustment.

Key properties:

- Direction: Horizontal or vertical stacking of child elements.
- Spacing between items: Fixed spacing or “auto” spacing.
- Padding: Space inside the container around content.
- Alignment: How children align within the auto layout frame (e.g., left, center, right, stretched).

Use cases:

- Buttons that grow/shrink as text label changes length.
- Lists that add or remove items without breaking alignment.
- Cards and sections that adapt to content and screen sizes.

Responsive behaviour:

- Resizing properties determine how components behave when the frame changes size (e.g., “Hug contents”, “Fill container”).
- Designers can combine auto layout with constraints and layout grids to create designs that map to responsive LCNC layouts and breakpoints.

11. Prototyping Concepts: Interactions, Transitions, and User Flows

Figma's prototyping features allow you to simulate how an app will behave without writing code.

Interactions

An interaction consists of:

- **Trigger:** What user action starts the interaction (e.g., On Tap, On Click, On Drag, While Hovering).
- **Action:** What happens (e.g., Navigate to, Open overlay, Scroll to, Change to another variant).
- **Target:** The destination frame or component.

Examples:

- Tapping a button moves to the next screen.
- Tapping a menu icon opens a sidebar overlay.
- Hovering over a card changes its variant to a highlighted state.

Transitions

Transitions define how the screen changes:

- Instant: Immediate switch with no animation.
- Dissolve: Fade between screens.
- Move In/Move Out/Push: Sliding transitions from different directions.
- Smart Animate: Animates differences in properties (position, size, opacity) between matching layers on two frames for more realistic animations.

User Flows

User flows are the paths users follow to complete tasks in the app. In Figma:

- Designers connect frames with arrows to represent navigation paths.
- They can define multiple flows (e.g., “Sign-up Flow”, “Checkout Flow”) and set separate starting points.

Value for LCNC:

- User flows serve as a blueprint for building workflows in LCNC tools.
- They help identify edge cases, loops, and error states before implementation.

12. Collaboration Features: Comments, Version History, Sharing

Comments

Comments allow stakeholders to discuss designs directly on the canvas:

- Users can pin comments to specific elements or areas of a frame.
- Mentions (e.g., @name) notify specific collaborators.
- Resolved comments can be closed to keep the canvas clean.

Version History

Figma maintains a version history of files:

- Named versions can be created at milestones (e.g., “v1.0 Approved UI”).
- Older versions can be inspected and restored if necessary.

This is particularly useful in LCNC projects where:

- Requirements evolve quickly.
- Multiple iterations need to be tracked for documentation and audits.

Sharing and Permissions

Sharing options:

- Link sharing with “Can view” or “Can edit” permissions.
- Organization-level controls in teams or workspaces.

13. Integration of Figma with Low-Code / No-Code Platforms

Many LCNC platforms support importing or aligning with Figma designs to speed up app creation.

Example: Bubble + Figma Integration

Bubble offers direct integration with Figma to import designs into Bubble apps:

- Designers create Figma files using a specific structure and naming conventions.
- Bubble's integration converts frames and components into Bubble elements and layout structures.
- This enables teams to move from prototype to functional web app without manual recreation of every element.

Example: Adalo

Adalo is a no-code app builder focused on web and mobile apps, including native iOS and Android.

Its community resources show examples of building Figma-like tools and design-heavy apps using Adalo's visual components and database.

Practical integration approaches:

- Direct Figma-to-platform import (for platforms that support it).
- Exporting design specs from Figma and manually reconstructing in LCNC tools.
- Using design tokens (colors, typography) defined in Figma as configuration values in LCNC themes or style settings.

Benefits:

- Reduced duplication of effort between design and build.
- Higher fidelity between prototype and final LCNC application.
- More efficient collaboration between designers and LCNC builders.

Sample Question Bank

- Define Figma in the context of LCNC applications.
- What is User Interface (UI)?
- What is User Experience (UX)?
- List the five phases of design thinking.
- State the four WCAG accessibility principles.
- What are frames in Figma?
- Define UI components in design systems.
- What is Auto Layout in Figma?
- Explain the role of Figma in LCNC application development.
- Differentiate between UI and UX with suitable examples.
- Describe the importance of design thinking in application design.
- Explain WCAG accessibility guidelines with examples.
- Describe benefits of cloud-based collaboration in Figma.
- Explain layout grids and their role in responsive design.
- Describe typography guidelines for mobile UI design.

- Explain concept of components and instances in Figma.
- Describe variants in UI component design.
- Explain concept of user flows in prototyping.
- Create a low-fidelity wireframe for a login screen.
- Apply design thinking process to design a student portal interface.
- Design a responsive layout using layout grids.
- Develop UI components for a dashboard screen.
- Apply accessibility principles while designing a form interface.
- Create auto layout design for a dynamic card component.
- Develop prototype interaction for navigation between screens.
- Apply typography hierarchy in designing a mobile application screen.
- Design a reusable button component with multiple variants.
- Create a simple user flow for online product purchase.
- Analyse importance of UI in improving usability of LCNC apps.
- Compare low-fidelity and high-fidelity prototypes.
- Analyse impact of poor UX on business performance.
- Examine benefits of using design systems in large projects.

- Analyze alignment and spacing practices in UI layouts.
- Compare auto layout with manual layout techniques.
- Analyze role of interaction triggers in prototype design.
- Examine importance of version history in collaborative design.
- Analyze responsive behaviour in UI design frameworks.
- Break down collaboration workflow in cloud-based design tools.
- Evaluate effectiveness of WCAG accessibility principles in modern apps.
- Justify use of Figma over traditional design tools.
- Evaluate benefits of component-based design approach.
- Assess importance of prototyping before LCNC implementation.
- Evaluate impact of real-time collaboration on project efficiency.
- Justify integration of Figma with LCNC platforms.
- Design a complete UI prototype for an e-commerce mobile application.
- Create a design system for a university management app.
- Develop an interactive prototype including transitions and animations.
- Design a responsive dashboard using auto layout and constraints.
- Create an accessible mobile app interface following WCAG principles.
- Develop a full user journey flow for an online service booking system.

Thank you all.....